Safe to Drop: an Exact Criterion for Quantum Temporaries

ANONYMOUS AUTHOR(S)

Quantum programming languages let a program discard a temporary register under sufficient conditions: a syntactic restriction on how the temporary was produced, a required final state, or the programmer's promise. We give the exact semantic condition behind these rules. A drop is *safe* when the program prefix that ends at it, with every register exposed, is observationally equivalent to a program that prepares the temporary in a fixed state, independently of the input, and leaves it idle: up to observational equivalence, the drop is a weakening. We analyze this definition into a decidable criterion. Without measurement, observational equivalence of programs that return every register is equality of denotations up to a global phase, and the programs that prepare the temporary independently denote exactly the maps $g \otimes W$, with g a unit vector on the temporary and W an isometry on the other registers. A drop is therefore safe exactly when the prefix's denotation K factors as $g \otimes W$, that is, when the slices $K_i = (\langle i| \otimes I)K$ along the temporary's computational basis, the Kraus operators of the channel the drop induces on the other registers, are pairwise proportional. With measurement, definition and criterion apply to the unnormalized map of each measurement record, so the verdict can depend on the outcome. The criterion extends to borrowed registers, and deciding it is coNP-complete for programs over X, CNOT and Toffoli, so no sound polynomial-time rule is complete for them unless P = NP. We implement it in a checker and a compiler to OpenQASM 3.

1 INTRODUCTION

Quantum programming languages let a program discard a temporary register when its scope ends, and each guards the discard with its own rule. Silq and Unqomp uncompute temporaries whose producers are `qfree` [1, 11]; Q#, Quipper, Qutes and ReQWIRE require the temporary to end in a designated state, on the programmer's word or by analysis [3, 5, 10, 12, 13], and fractional types require the same of reversible programs [2]. Each rule is a sufficient condition for a single semantic property: the program behaves as if the temporary had been prepared on its own and left idle. We define this property by observational equivalence and analyze the definition into an exact criterion that a compiler can check. Exactness has a price: deciding the criterion is coNP-complete for programs over X, CNOT and Toffoli, so every sound polynomial-time rule rejects some safe drops unless P = NP.

We write $\Omega \vdash M : A$ for a program with input registers Ω and output type A. Without measurement it denotes an isometry $[\![M]\!] : [\![\Omega]\!] \to \mathcal{H}_{\mathsf{hid}} \otimes [\![A]\!]$, where $\mathcal{H}_{\mathsf{hid}}$ holds the registers M has dropped: `drop` ends a register's scope and leaves the state unchanged. A closed context C supplies the inputs of a program and may compute further on its outputs; the observable result of a closed program is the probability of each measurement record and of each standard-basis outcome of its output. Two programs of the same type are *observationally equivalent*, $M \cong M'$, when $C[M]$ and $C[M']$ have the same observable result for every closed context C. Now fix a drop site s in M whose block v is a tuple of temporaries allocated in the state ψ, and let

$P_1 = M|_s$ be the prefix of M before s with every `drop` deleted, so that P_1 returns every register; a context observes a register's correlations only while the register is in scope. The drop is *safe* when $P_1 \cong P_2$ for some *witness*

$$P_2 \;=\; \texttt{new}\ v\ @\ \psi;\;\; R\ v;\;\; Q,$$

with R a unitary on the block and Q a program that acts only on the other registers. In P_2 the block is prepared on its own and left idle, so dropping it is a weakening: safety says that the drop is a weakening up to observational equivalence. Without measurement, programs that return every register are observationally equivalent exactly when their denotations agree up to a global phase, which Q can absorb. Safety thus asks for

$$[\![P_1]\!] \;=\; [\![P_2]\!].$$

Example 1.1 gives three well-typed programs. Each allocates t in $|0\rangle$, computes, and drops t while returning v; purple marks the lines that (b) and (c) add to (a).

Example 1.1 (the running examples).

(a) *superpose, then drop*

```
prog a (v : [2]) : [2] {
  new t : [2] @ |0>;
  H t;


  drop t;
  return v }
```

(b) *superpose, then read*

```
prog b (v : [2]) : [2] {
  new t : [2] @ |0>;
  H t;
  CX t v;


  drop t;
  return v }
```

(c) *copy, measure, one joint gate*

```
prog c (v : [2]) : [2] {
  new t : [2] @ |0>;
  H v;  CX v t;
  p = lift[{0}] t;
  V t v;
  drop t;
  return v }
```

Program (a) is safe: its prefix already has the shape of P_2, with $R = H$. In program (b), the `CX` entangles t with v at input $|0\rangle_v$, whereas every P_2 returns t in the product state $R\psi$, so its drop is unsafe. The temporary is dead after the drop in both programs; scope information alone cannot decide safety. Program (c), treated in section 3, is safe on one measurement outcome and unsafe on the other. The definition is semantic: the prefix of Deutsch–Jozsa applies the oracle to its temporary, yet it is observationally equivalent to a P_2 that prepares the temporary in $|-\rangle$ and applies the phase oracle to the query register alone.

The definition quantifies over all contexts and all witnesses P_2; analyzing it reduces safety to a property of one linear map. Let $K = [\![P_1]\!] : [\![\Omega]\!] \to \mathcal{H}_v \otimes \mathcal{H}_{\mathsf{rest}}$, where $\mathcal{H}_{\mathsf{rest}}$ holds every register outside the block, including those dropped earlier. Unfolding the witness gives $[\![P_2]\!]x = R\psi \otimes [\![Q]\!]x$, so safety asks for $K = g \otimes W$ with $g = R\psi$ and $W = [\![Q]\!]$. Conversely, since our gates include every unitary, each unit vector g is $R\psi$ for some R, and each isometry W is $[\![Q]\!]$ for some Q that extends W to a unitary. Expanding K along the computational basis of the block, with slices $K_i = (\langle i| \otimes I)K$, then gives

$$\text{the drop is safe} \iff K = g \otimes W \iff K_i = g_i\, W \text{ for every } i \iff \operatorname{rank}\Gamma \le 1,$$

where $\Gamma_{ij} = \mathrm{tr}(K_i^\dagger K_j)$ is the Gram matrix of the slices. The first equivalence is theorem 2.1; the others are linear algebra. The slices are the Kraus operators of the channel $\rho \mapsto \sum_i K_i \rho K_i^\dagger$ that the drop induces on the other registers, so a drop is safe exactly when this channel has Kraus rank one. The unit vector g, the *certificate*, is the state in which the block ends at every input. Program (a) has $K_0 = K_1 = I/\sqrt{2}$ and $g = |+\rangle$; program (b) has $K_0 = I/\sqrt{2}$ and $K_1 = X/\sqrt{2}$, which are linearly independent. The criterion accepts (a), whose producer H lies outside `qfree`, and the temporary of Deutsch–Jozsa, which ends in $|-\rangle$ for every oracle, whereas designated-state rules require $|0\rangle$.

With measurement, a program denotes one unnormalized map per measurement record, and observational equivalence compares the probability of each record as well. Safety is defined record by record, against witnesses that may realize the remaining computation with one extra qubit and a measurement of which they keep one outcome. The same analysis shows that the drop is safe on a record o exactly when the prefix's map on o factors as $\sigma_s^o = g \otimes W_o$ with W_o a contraction, where $\|W_o x\|^2$ is the probability of o at a unit input x (theorem 3.1). Program (c) is safe on exactly one of its two records. Coarser semantics cannot decide safety: two programs can share their output channel and record probabilities while receiving opposite verdicts on every record (section 4).

For borrowed registers, which must return to the caller with their entanglement intact, the certificate generalizes to an isometry B on the borrowed inputs, $K = B \otimes W$. A safe block is in a known state, so a compiler can reclaim it, and reusing reclaimed temporaries preserves every record, its probability and the reduced output state. We prove type safety and these results for a typed first-order language with finite-dimensional registers, measurement and classical conditionals, and implement the criterion in Haskell, with exact arithmetic for the verdicts and an OpenQASM 3 back end that emits reclaiming gates (section 4).

2 SEPARATING THE TEMPORARY FROM THE COMPUTATION

For the safety test, we will call one occurrence of `drop` a *site*, and the tuple of registers it names its *block*. Here each block is just t. A joint pure state of n qubits is a unit vector with 2^n complex coordinates, one per bit string. For programs (a) and (b), let K denote the prefix's state map just before the drop. It maps an input x in v to the joint state Kx of t and v. Put t first in each tensor product. Program (a) gives

$$x \xrightarrow{\texttt{new t @ |0>}} |0\rangle \otimes x \xrightarrow{\texttt{H t}} \frac{|0\rangle \otimes x + |1\rangle \otimes x}{\sqrt{2}} = |+\rangle \otimes x.$$

Similarly, program (b) has the same first two steps, followed by

$$|+\rangle \otimes x \xrightarrow{\texttt{CX t v}} \frac{|0\rangle \otimes x + |1\rangle \otimes Xx}{\sqrt{2}}.$$

Program (a) leaves v alone for both temporary labels. Program (b) leaves v alone for label 0 and applies the bit flip X for label 1.